Assignment 2 - PS04: Hex Game AI using MCTS

Design Document | Group G164 | AIMLCZG557/AECLZG557

1. Objective and game model

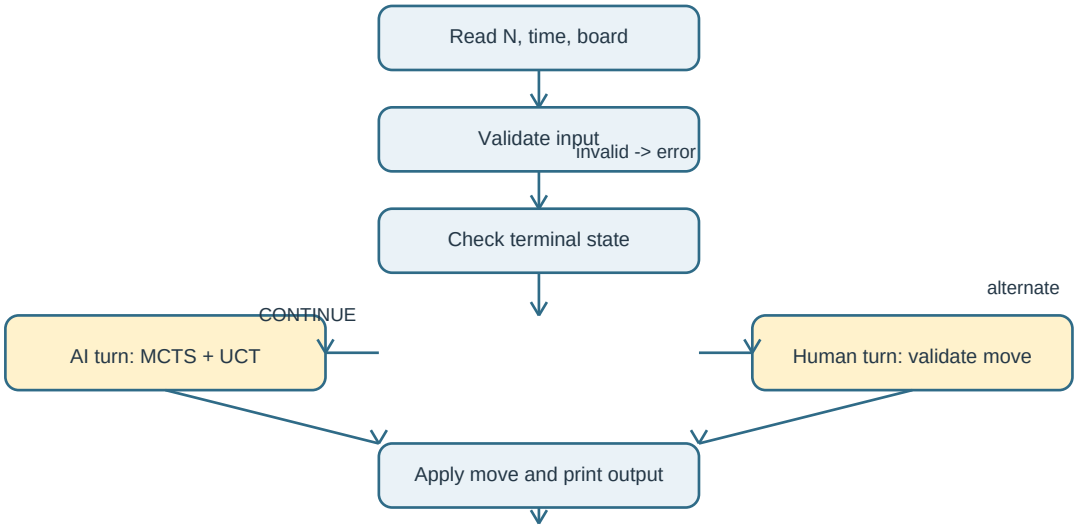
This program plays an N x N Hex game for 7 <= N <= 11. Player A (1) is the AI and must connect the top row to the bottom row. Player B (2) is the human and must connect the left column to the right column. Empty cells contain 0. The AI chooses a legal move within the input time limit and the game ends immediately after a winning connection.

Board representation and adjacency

The board is a two-dimensional list. Each cell has exactly six Hex neighbors: top, bottom, left, right, top-right (r-1,c+1), and bottom-left (r+1,c-1). Top-left and bottom-right are forbidden. Bounds are checked for every generated neighbor.

```
DIRECTIONS = [(-1,0), (1,0), (0,-1), (0,1), (-1,1), (1,-1)]
```

The end-to-end program flow is:

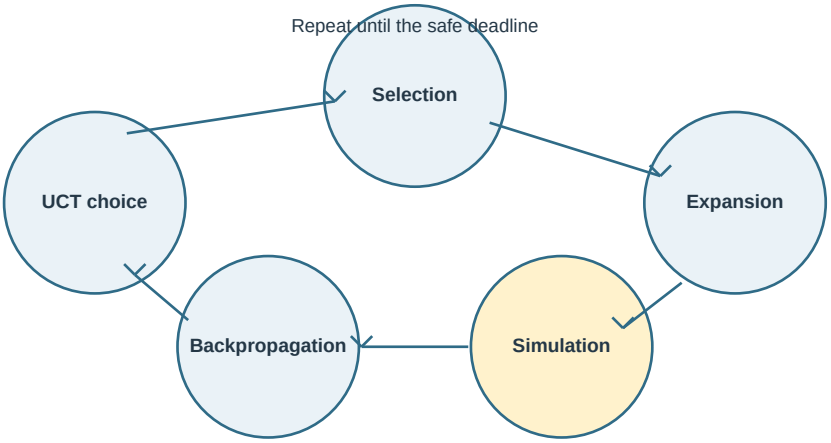


Implementation mapping

HexBoard manages state, legal moves, cloning, and DFS winner detection. MCTSNode stores a search state. MCTSAgent performs UCT search. The parser validates inputPS04.txt, the game loop alternates turns, and OutputWriter produces outputPS04.txt.

2. MCTS and UCT design

A root node is created from the current board. The four MCTS stages are repeated until a safe deadline. The final move is the most visited root child. An immediate winning move is taken directly when found, which is a tactical optimization around the MCTS search.



Selection and UCT

Selection follows fully expanded, non-terminal children. An unvisited child receives infinity and is explored first. Rewards are stored from the root AI perspective. At a Player A node, exploitation is the child root win rate; at a Player B node, it is one minus that win rate, so the opponent's preference is handled correctly.

```
UCT = exploitation + C * sqrt(ln(parent_visits) / child_visits)
C = sqrt(2)
Player A turn: exploitation = wins / visits
Player B turn: exploitation = 1 - (wins / visits)
```

Expansion, simulation, and backpropagation

Expansion removes one untried legal move and creates a child using a cloned board. Terminal nodes are never expanded. Simulation alternates players, checks for a winner, and uses a shuffled rollout policy that prefers an immediate winning move. Backpropagation increments visits on every ancestor and adds 1 for an AI win, 0 for a human win, or 0.5 for the defensive no-winner fallback.

Time control

time.perf_counter() controls the search. The program reserves a safety margin of 1% of the requested time, capped at 25 ms, for final move selection and output. The rollout also checks the deadline so a long simulation cannot cause an avoidable overrun.

Core pseudocode

```
while current_time < deadline:
    node = select_uct(root)
    if node is non_terminal and has_untried_moves:
        node = expand(node)
    winner = rollout(node.state)
    backpropagate(node, winner)
return child_with_most_visits(root)
```

3. Correctness, validation, and complexity

Winner detection uses DFS with a visited set. Player A starts from all matching cells in row 0 and succeeds at row N-1. Player B starts from all matching cells in column 0 and succeeds at column N-1. The same six-neighbor function is used by game logic and rollouts, preventing accidental eight-neighbor connectivity.

Error scenarios

Scenario	Handling
N outside 7..11	Reject with a clear validation error.
Non-positive/non-numeric time	Reject before starting the game.
Wrong board-cell count	Require exactly N squared values.
Cell not in 0,1,2	Reject as malformed board input.
Bad human syntax/coordinates	Reprompt without crashing.
Occupied human cell	Reject and reprompt.
EOF or missing file	Exit/report gracefully.
Terminal initial board	Report winner without requesting a move.
No legal AI move	End safely rather than selecting an invalid cell.

Required tests and observed run

Test	Result
Six-neighbor rule	Forbidden diagonals excluded; allowed diagonals included.
Player A and B DFS paths	Both connection directions detected.
Invalid occupied/out-of-range moves	Rejected.
Immediate AI winning move	AI selects a legal winning move.
7 x 7 empty-board run	Player A wins after 19 turns; 10 AI and 9 human moves; average AI time 1,782.15 ms; maximum depth 49; maximum expan

Complexity

DFS/BFS winner detection costs O(N squared) time and O(N squared) space. If a rollout has depth D and checks the board after each move, one rollout is approximately O(DN squared). With I MCTS iterations, total search work is approximately O(I D N squared). I is controlled by the wall-clock budget, so the number of completed iterations varies by machine. Full-board copies make tree memory approximately O(I N squared).

4. Alternate model, scalability, and AI context

Alternate model: rollback Union-Find

An alternate representation treats cells as graph nodes and adds virtual boundary nodes. Player A boundary cells connect to top/bottom virtual nodes; Player B boundary cells connect to left/right virtual nodes. A win is detected when the matching virtual nodes become connected. Union-Find provides almost constant amortized connectivity checks, O(alpha(N squared)), but MCTS needs rollback support or state copying when hypothetical branches are explored. The current matrix plus DFS model is easier to audit and is appropriate for N <= 11.

Boards larger than 11 x 11

The program can stop after a deterministic wall-clock limit, but it cannot guarantee a deterministic number of iterations or the exact best move across different machines. Larger state spaces make plain random rollouts less informative. Suitable approaches are transposition tables with Zobrist hashing, progressive widening, RAVE/AMAF, parallel MCTS, and neural policy/value guidance. They reduce repeated work or focus computation on promising moves.

Game theory in present AI

Silver et al., 'A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play' (Nature, 2018), combines self-play reinforcement learning with tree search. A policy guides promising actions and a value estimates outcomes in an adversarial zero-sum setting. The same strategic structure appears here: each rollout models competing players and the AI maximizes its estimated chance of winning. The idea extends to multi-agent reinforcement learning and LLM agents that negotiate or compete. In fake-news generation and detection, game-theoretic framing can model an adversary adapting to a detector; robust training must therefore consider changing strategies rather than a fixed classifier. The limitation is that reward design, opponent modeling, and equilibrium assumptions may not match real-world behavior.

Trade-offs and submission checklist

Choice	Benefit	Cost
MCTS + UCT	Time-bounded and scalable to large branching spaces.	Sampling is stochastic and not an exact guarantee.
DFS winner detection	Simple, transparent, easy to verify.	Repeated board scans reduce rollout speed.
Full board cloning	Safe branch isolation and simple code.	Higher memory and copying cost.
Rollback Union-Find	Very fast incremental connectivity.	More complex undo logic in MCTS.
Randomized rollouts	Cheap and easy to implement.	Can miss strategic threats without tactical checks.

The final G164 package contains designPS04_G164.pdf, G164_Contribution.xlsx, inputPS04.txt, outputPS04.txt, and hexGamePS04.py. The self-test command is: python hexGamePS04.py --test

Reference: Silver, D. et al. (2018). Nature, 550, 354-359.